OpenGL实验报告

OpenGL实验报告
《计算机图形学》实验课程大作业报告：地球仪的绘制
一、实验目的
二、实验内容与要求
基本要求
附加要求
三、主要函数和功能
1. `handleKeyboardInput` 函数
2. `initializeLighting` 函数
3. `isPowerOfTwo` 函数
4. `loadBMPTexture` 函数
5. `initializeRendering` 函数
6. `deinitializeRendering` 函数
7. `handleWindowResize` 函数
8. `drawEarth` 函数
9. `drawStandPole` 函数
10. `drawStandBase` 函数
11. `drawScene` 函数
12. `update` 函数
13. `handleMouseMotion` 函数
14. `handleMouseClick` 函数
四、主要代码
实验思路和步骤
实验思路
实验步骤
结果
心得
完整代码

《计算机图形学》实验课程大作业报告：地球仪的绘制

一、实验目的

- 掌握计算机图形学中的 3D 图形绘制技术，利用 OpenGL 库创建和操作 3D 物体，绘制出一个逼真的地球仪模型。
- 深入理解纹理映射的原理和实现方法，将 2D 世界地图纹理无缝地贴到 3D 圆球上，使地球表面呈现出真实的地理特征。
- 学会添加光照和材质效果，通过合理的光照设置和材质属性配置，增强地球仪的立体感和真实感。
- 实现用户交互功能，包括双点触控缩放球体和拖动旋转球体，提升用户的交互体验，使用户能够更直观地观察地球仪的各个部分。

- 进一步探索如何通过添加支架和光照阴影等效果，提高 3D 场景的逼真度，使地球仪模型更加完整和真实。

二、实验内容与要求

基本要求

1. 绘制圆球并实现纹理映射
    - 使用 OpenGL 绘制一个圆球，作为地球仪的基础模型。
    - 加载世界地图作为纹理，将其无缝地映射到圆球表面，可以使用已有的世界地图纹理文件或自行寻找合适的纹理文件，确保纹理贴图没有明显缝隙，使地球表面的地理特征清晰可见。
2. 添加光照和材质效果
    - 为场景添加简单的光照效果，设置合适的光源参数，使圆球在光照下呈现出立体的外观，增强视觉效果。
    - 为圆球设置合适的材质属性，如光泽度、反射特性等，使其与光照效果相匹配，进一步提升地球仪的真实感。
3. 实现用户交互功能
    - 实现双点触控缩放功能，允许用户通过触控操作（模拟鼠标操作）放大或缩小圆球，方便用户观察地球仪的细节。
    - 实现拖动旋转功能，使用户可以通过鼠标拖动来旋转圆球，自由切换观察角度，全方位地查看地球仪。

附加要求

1. 添加支架
    - 为地球仪添加一个真实的支架，使其更具真实感。支架可以包括底座和支撑杆，通过绘制圆柱体和圆盘等几何形状来实现，确保支架与地球仪的比例协调，稳固支撑地球仪。
2. 增强光照和阴影效果
    - 增加更复杂的光照效果，模拟真实的光照场景，如多光源照明、光源移动等，使地球仪在不同光照条件下的视觉效果更加丰富和真实。
    - 尝试添加阴影效果，计算光源与地球仪之间的遮挡关系，生成相应的阴影，使地球仪的 3D 场景更加逼真，增强空间感和立体感。

三、主要函数和功能

1. `handleKeyboardInput` 函数

```
void handleKeyboardInput(unsigned char key, int x, int y) {
    switch (key) {
    case 27: // Escape 键
```

```
        exit(0);
        break;
    case '+':
        radius += 10;
        glutPostRedisplay();
        break;
    case '-':
        radius -= 10;
        glutPostRedisplay();
        break;
    }
}
```

- 功能：处理键盘输入事件。
  - 按下 `Esc` 键时，程序退出。
  - 按下 `+` 键，增大地球的半径 `radius` 并触发重新绘制，实现放大效果。
  - 按下 `-` 键，减小地球的半径 `radius` 并触发重新绘制，实现缩小效果。

2. `initializeLighting` 函数

```
void initializeLighting() {
    GLfloat ambientLight[] = { 0.2, 0.2, 0.2, 1 };
    GLfloat diffuseLight[] = { 0.8, 0.8, 0.8, 1 };
    GLfloat specularLight[] = { 0.5, 0.5, 0.5, 1 };
    GLfloat lightPosition[] = { 400, 240, 1, 1 };
    GLfloat specularReflection[] = { 1, 1, 1, 1 };

    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ambientLight);
    glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight);
    glLightfv(GL_LIGHT0, GL_SPECULAR, specularLight);

    glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 128);
}
```

- 功能：初始化光照设置。
  - 设置环境光、漫反射光、镜面反射光的参数，定义光源的位置和材质的光泽度。
  - 使物体表面在光照下具有反光效果，增强立体感和真实感。

3. `isPowerOfTwo` 函数

```
int isPowerOfTwo(int n) {
    if (n <= 0)
        return 0;
    return (n & (n - 1)) == 0;
}
```

- 功能：判断一个整数是否为 2 的整数次幂。

- 用于检查纹理图像的尺寸是否符合 OpenGL 纹理要求，确保纹理能够正确加载和应用。

4. `loadBMPTexture` 函数

```
GLuint loadBMPTexture(const char* fileName) {
    // 函数实现省略，具体功能如下：
    // 从 BMP 文件中读取纹理数据，处理纹理尺寸（确保为 2 的整数次幂）。
    // 生成纹理对象，设置纹理参数，将纹理数据加载到纹理对象中，并返回纹理编号。
}
```

- 功能：加载纹理。
  - 从 BMP 文件中读取纹理数据，处理纹理尺寸，生成纹理对象并设置纹理参数，使纹理能够正确应用到 3D 物体上。

5. `initializeRendering` 函数

```
void initializeRendering() {
    earth = gluNewQuadric();
    initializeLighting();
    glEnable(GL_LIGHT0);
    glEnable(GL_LIGHTING);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_LINE_SMOOTH);
}
```

- 功能：初始化渲染环境。
  - 创建用于绘制地球的二次曲面对象，调用 `initializeLighting` 函数设置光照，启用光照、深度测试和线平滑等 OpenGL 特性，为后续的渲染做好准备。

6. `deinitializeRendering` 函数

```
void deinitializeRendering() {
    gluDeleteQuadric(earth);
}
```

- 功能：清理渲染资源。
  - 删除用于绘制地球的二次曲面对象，释放相关资源，避免内存泄漏。

7. `handleWindowResize` 函数

```
void handleWindowResize(int w, int h) {
    if (h == 0)
        h = 1;
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(0, w, 0, h, -200, 200);
    glMatrixMode(GL_MODELVIEW);
}
```

- 功能：处理窗口大小调整事件。
  - 设置视口和投影矩阵，使用正交投影，确保在窗口大小改变时，渲染的场景能够正确显示。

8. `drawEarth` 函数

```
void drawEarth() {
    //具体功能如下：
    // 绑定纹理并设置纹理生成模式，启用纹理，旋转并绘制地球球体。
}
```

- 功能：绘制地球。
  - 绑定纹理，设置纹理生成模式为球面映射，启用纹理，旋转地球并绘制球体，使地球表面呈现出真实的地理特征。

9. `drawStandPole` 函数

```
void drawStandPole() {
    // 具体功能如下：
    // 平移和旋转矩阵，绘制圆柱体作为支撑杆。
}
```

- 功能：绘制地球仪的支撑杆。
  - 通过平移和旋转矩阵，绘制一个圆柱体作为地球仪的支撑杆，使其与地球仪的比例协调，稳固支撑地球仪。

10. `drawStandBase` 函数

```
void drawStandBase() {
    //具体功能如下：
    // 旋转矩阵，绘制圆盘作为底座。
}
```

- 功能

  ：绘制地球仪的底座。
  - 使用旋转矩阵，绘制一个圆盘作为地球仪的底座，使地球仪更加完整和真实。

11. `drawScene` 函数

```
void drawScene() {
    // 具体功能如下：
    // 清除缓冲区，进行矩阵变换，绘制底座、支撑杆和地球，并实现自动旋转和用户交互功能。
}
```

- 功能：绘制整个场景。
  - 清除缓冲区，设置背景颜色，进行矩阵变换，包括缩放、平移和旋转，按顺序绘制底座、支撑杆和地球，并实现自动旋转和用户交互功能，如缩放和旋转地球。

12. `update` 函数

```
void update(int value) {
    angle += 2.0f;
    if (angle > 360) {
        angle -= 360;
    }
    glutPostRedisplay();
    glutTimerFunc(16, update, 0);
}
```

- 功能：实现自动旋转效果。
  - 不断更新地球的旋转角度，循环角度在 0 到 360 度之间，触发重绘并使用定时器循环更新，使地球能够自动旋转。

13. `handleMouseMotion` 函数

```
void handleMouseMotion(int x, int y) {
    int rotationXDelta = x - oldMouseX;
    int rotationYDelta = 480 - y - oldMouseY;
    rotationX += rotationXDelta;
    rotationY += rotationYDelta;
    glutPostRedisplay();
}
```

- 功能：处理鼠标拖动旋转事件。
  - 计算鼠标拖动的偏移量，更新旋转角度并触发重绘，使用户可以通过鼠标拖动来旋转地球，自由切换观察角度。

14. `handleMouseClick` 函数

```
void handleMouseClick(int button, int state, int x, int y) {
    static bool done = false;
    if (button == GLUT_LEFT_BUTTON) {
        oldMouseX = x;
        oldMouseY = 480 - y;
    }
}
```

- 功能：处理鼠标点击事件。
  - 记录鼠标点击位置，用于后续的拖动操作，使用户可以通过鼠标点击来开始拖动旋转地球。

四、主要代码

```
#include <windows.h>
#include <GL/glut.h>
#include <cmath>
#include <cstdio>
#include <cstdlib>

using namespace std;

// 角度相关的静态变量
static float angle = 30.0f;
static float cameraAngle = 0.0f;

// 鼠标操作相关的静态变量
static int oldMouseX;
static int oldMouseY;
static float rotationX = 0;
static float rotationY = 0;

// 位置相关的静态变量
static float position[3] = { 1.0f, 1.0f, 1.0f };
static float radius = 100;

// 地球的二次曲面对象
static GLUquadric* earth;

// BMP 头文件的长度
#define BMP_HEADER_LENGTH 54

// 纹理对象编号
GLuint earthTexture;

// 处理按键操作
void handleKeyboardInput(unsigned char key, int x, int y) {
    // 函数实现省略
}

// 初始化光照
void initializeLighting() {
    // 函数实现省略
}
```

```
// 判断一个整数是否是 2 的整数次幂
int isPowerOfTwo(int n) {
    // 函数实现省略
}

// 加载纹理
GLuint loadBMPTexture(const char* fileName) {
    // 函数实现省略
}

// 初始化渲染相关设置
void initializeRendering() {
    // 函数实现省略
}

// 清除渲染资源
void deinitializeRendering() {
    // 函数实现省略
}

// 处理窗口大小调整
void handleWindowResize(int w, int h) {
    // 函数实现省略
}

// 绘制地球
void drawEarth() {
    // 函数实现省略
}

// 绘制支架的杆子
void drawStandPole() {
    // 函数实现省略
}

// 绘制地球外框
void drawStandBase() {
    // 函数实现省略
}

// 绘制场景
void drawScene() {
    // 函数实现省略
}

// 实现自动旋转效果
void update(int value) {
    // 函数实现省略
}

// 处理鼠标拖动旋转事件
void handleMouseMotion(int x, int y) {
    // 函数实现省略
}

// 处理鼠标点击事件
```

```
void handleMouseClick(int button, int state, int x, int y) {
    // 函数实现省略
}

int main(int argc, char** argv) {
    // 初始化 GLUT，设置显示模式和窗口大小，创建窗口，初始化渲染，加载纹理，注册各种回调函数，进入主循环并在结束时清理资源
}
```

实验思路和步骤

实验思路

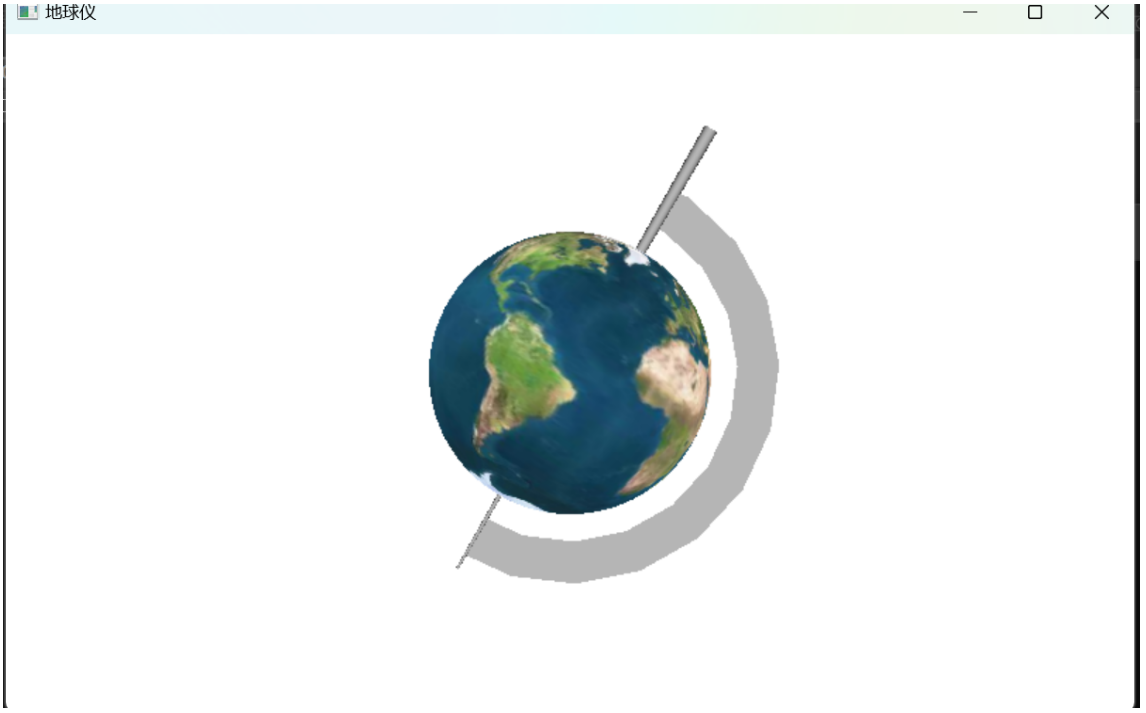
- 首先，使用 OpenGL 和 GLUT 库创建一个窗口，并初始化显示模式和渲染环境，为后续的图形绘制做好准备。
- 加载世界地图纹理，确保纹理尺寸符合 OpenGL 的要求，以便能够正确地应用到地球仪模型上。
- 绘制地球，使用二次曲面对象和纹理映射技术将纹理贴到球面上，同时设置光照和材质效果，使地球呈现出立体的外观和真实的地理特征。
- 为地球仪添加支架，通过绘制圆柱体和圆盘等几何形状来实现，使地球仪更具真实感和完整性。
- 实现用户交互功能，包括键盘输入控制缩放、鼠标拖动控制旋转和自动旋转，使用户能够更直观地观察和操作地球仪。
- 对于附加要求，进一步优化光照和阴影效果，模拟真实的光照场景，使地球仪的 3D 场景更加逼真和生动。

实验步骤

1. 环境搭建：
    - 包含所需的头文件，包括 Windows 系统头文件和 OpenGL、GLUT 库的头文件。
    - 定义全局变量存储程序状态，如旋转角度、缩放因子、纹理对象等。
2. 初始化：
    - 在 `initializeRendering` 函数中，创建二次曲面对象，调用 `initializeLighting` 函数设置光照，启用相关 OpenGL 特性。
    - 在 `loadBMPTexture` 函数中，从文件加载世界地图纹理，处理纹理尺寸并生成纹理对象，确保纹理能够正确应用到地球仪模型上。
3. 场景绘制：
    - 在 `drawScene` 函数中，先清除缓冲区，设置背景颜色和矩阵变换，绘制地球仪的底座、支撑杆和地球。
    - 调用 `drawEarth` 函数绘制地球，使用 `gluSphere` 绘制球体并应用纹理，使地球表面呈现出真实的地理特征。
    - 调用 `drawStandPole` 和 `drawStandBase` 函数绘制支架，通过绘制圆柱体和圆盘来实现。
4. 用户交互：
    - 使用 `handleKeyboardInput` 函数处理键盘输入，实现通过键盘控制缩放地球的功能。
    - 使用 `handleMouseClick` 和 `handleMouseMotion` 函数处理鼠标操作，实现通过鼠标拖动旋转地球的功能。
    - 使用 `update` 函数实现自动旋转效果，通过定时器循环更新地球的旋转角度并触发重绘。

结果

- 运行程序后，将看到一个带有世界地图纹理的地球仪，它可以自动旋转。用户可以通过键盘缩放地球，通过鼠标拖动旋转地球。同时，地球仪有一个简单的支架支撑，并且有基本的光照效果。



心得

- 通过本实验，对 OpenGL 图形编程有了更深入的理解，尤其是 3D 图形的绘制和变换操作，如平移、旋转和缩放等，掌握了如何使用 OpenGL 库创建和操作 3D 物体。
- 掌握了纹理映射的实现细节，包括纹理的加载、处理和应用，以及如何确保纹理无缝映射到 3D 物体上，使地球表面呈现出真实的地理特征。
- 实现用户交互功能让 3D 场景更具动态性和交互性，理解了如何处理鼠标和键盘事件，提升了用户的交互体验。

- 在添加光照和材质时，学会了如何设置不同的光照参数和材质属性，以增强 3D 物体的真实感和立体感。
- 对于附加要求中的支架绘制和光照阴影添加，需要进一步探索 OpenGL 的高级特性，这也让我认识到图形学的复杂性和可扩展性。在处理阴影时，需要考虑更多的光线传播和遮挡问题，这是后续可以继续深入学习和研究的方向。

完整代码

```
#include <windows.h>
#include <GL/glut.h>
#include <cmath>
#include <cstdio>
#include <cstdlib>

using namespace std;

// 角度相关的静态变量
static float angle = 30.0f;
static float cameraAngle = 0.0f;

// 鼠标操作相关的静态变量
static int oldMouseX;
static int oldMouseY;
static float rotationX = 0;
static float rotationY = 0;

// 位置相关的静态变量
static float position[3] = { 1.0f, 1.0f, 1.0f };
static float radius = 100;

// 地球的二次曲面对象
static GLUquadric* earth;

// BMP 头文件的长度
#define BMP_HEADER_LENGTH 54

// 纹理对象编号
GLuint earthTexture;

// 处理按键操作
void handleKeyboardInput(unsigned char key, int x, int y) {
    switch (key) {
    case 27: // Escape 键
        exit(0);
        break;
    case '+':
        radius += 10;
        glutPostRedisplay();
        break;
    case '-':
        radius -= 10;
        glutPostRedisplay();
        break;
    }
```

```
}

// 初始化光照
void initializeLighting()
{
    GLfloat ambientLight[] = { 0.2, 0.2, 0.2, 1 };
    GLfloat diffuseLight[] = { 0.8, 0.8, 0.8, 1 };
    GLfloat specularLight[] = { 0.5, 0.5, 0.5, 1 };
    GLfloat lightPosition[] = { 400, 240, 1, 1 };
    GLfloat specularReflection[] = { 1, 1, 1, 1 };

    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ambientLight);
    glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight);
    glLightfv(GL_LIGHT0, GL_SPECULAR, specularLight);

    glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 128);
}


// 判断一个整数是否是 2 的整数次幂
int isPowerOfTwo(int n)
{
    if (n <= 0)
        return 0;
    return (n & (n - 1)) == 0;
}


// 加载纹理
/* 读取一个 BMP 文件作为纹理
* 若失败，返回 0；若成功，返回纹理编号
*/
GLuint loadBMPTexture(const char* fileName)
{
    GLint width, height, totalBytes;
    GLubyte* pixels = 0;
    GLuint textureID = 0;
    GLint lastTextureID = 0; // 修改为 GLint 类型

    // 打开文件
    FILE* file;
    int err = fopen_s(&file, fileName, "rb");
    if (err != 0) { return 0; }

    // 读取文件中图像的宽度和高度
    fseek(file, 0x0012, SEEK_SET);
    fread(&width, 4, 1, file);
    fread(&height, 4, 1, file);
    fseek(file, BMP_HEADER_LENGTH, SEEK_SET);

    // 计算每行像素所占字节数及总字节数
    {
        GLint lineBytes = width * 3;
        while (lineBytes % 4 != 0)
            ++lineBytes;
```

```
        totalBytes = lineBytes * height;
    }

    // 根据总字节数分配内存
    pixels = (GLubyte*)malloc(totalBytes);
    if (pixels == 0)
    {
        fclose(file);
        return 0;
    }

    // 读取像素数据
    if (fread(pixels, totalBytes, 1, file) <= 0)
    {
        free(pixels);
        fclose(file);
        return 0;
    }

    // 对于旧版本的兼容处理，如果图像宽高不是 2 的整数次幂或超过 OpenGL 最大值，则进行缩放
    {
        GLint max;
        glGetIntegerv(GL_MAX_TEXTURE_SIZE, &max);
        if (!isPowerOfTwo(width)
            || !isPowerOfTwo(height)
            || width > max
            || height > max)
        {
            const GLint newWidth = 256;
            const GLint newHeight = 256; // 缩放后的新大小为边长的正方形
            GLint newLineBytes, newTotalBytes;
            GLubyte* newPixels = 0;

            // 计算新的每行字节数和总字节数
            newLineBytes = newWidth * 3;
            while (newLineBytes % 4 != 0)
                ++newLineBytes;
            newTotalBytes = newLineBytes * newHeight;

            // 分配内存
            newPixels = (GLubyte*)malloc(newTotalBytes);
            if (newPixels == 0)
            {
                free(pixels);
                fclose(file);
                return 0;
            }

            // 进行像素缩放
            gluScaleImage(GL_RGB,
                width, height, GL_UNSIGNED_BYTE, pixels,
                newWidth, newHeight, GL_UNSIGNED_BYTE, newPixels);

            // 释放原像素数据，更新 pixels 并设置新的宽高
            free(pixels);
            pixels = newPixels;
```

```
            width = newWidth;
            height = newHeight;
        }
    }

    // 分配一个新的纹理编号
    glGenTextures(1, &textureID);
    if (textureID == 0)
    {
        free(pixels);
        fclose(file);
        return 0;
    }

    // 绑定新纹理，载入并设置纹理参数
    glGetIntegerv(GL_TEXTURE_BINDING_2D, &lastTextureID);
    glBindTexture(GL_TEXTURE_2D, textureID);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_REPLACE);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_BGR_EXT,
GL_UNSIGNED_BYTE, pixels);
    glBindTexture(GL_TEXTURE_2D, lastTextureID);  // 恢复之前的纹理绑定
    free(pixels);
    return textureID;
}


// 初始化渲染相关设置
void initializeRendering() {
    // 创建地球的二次曲面对象
    earth = gluNewQuadric();

    // 初始化光照
    initializeLighting();

    glEnable(GL_LIGHT0);
    glEnable(GL_LIGHTING);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_LINE_SMOOTH);
}


// 清除渲染资源
void deinitializeRendering()
{
    gluDeleteQuadric(earth);
}


// 处理窗口大小调整
void handleWindowResize(int w, int h) {
    if (h == 0)
    {
```

```
        h = 1;
    }
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(0, w, 0, h, -200, 200);
    glMatrixMode(GL_MODELVIEW);
}


// 绘制地球
void drawEarth()
{
    static bool textureLoaded = false;
    // 绑定纹理到目标
    glBindTexture(GL_TEXTURE_2D, earthTexture);
    if (!textureLoaded)
    {
        // 纹理坐标自动生成
        glTexGeni(GL_S, GL_TEXTURE_GEN_MODE, GL_SPHERE_MAP);
        glTexGeni(GL_T, GL_TEXTURE_GEN_MODE, GL_SPHERE_MAP);
        // 表面生成纹理坐标
        gluQuadricDrawStyle(earth, GL_FILL);
        gluQuadricNormals(earth, GLU_SMOOTH);
        gluQuadricTexture(earth, GL_TRUE);
    }
    // 生成球体
    glPushMatrix();
    {
        glEnable(GL_TEXTURE_2D);
        glRotatef(-90, 1, 0, 0);
        gluSphere(earth, radius, 100, 100);
        glDisable(GL_TEXTURE_2D);
    }
    glPopMatrix();
}


// 绘制支架的杆子
void drawStandPole() {
    glPushMatrix();
    glTranslatef(0, 200, 0);  // 将杆子起点移动到合适位置
    glRotatef(90, 1, 0, 0);  // 旋转杆子使其垂直

    GLUquadric* pole = gluNewQuadric();
    gluCylinder(pole, 5, 1, 160 + 2 * radius, 20, 20);  // 绘制圆柱体作为杆子，半径为
5，高度为 400

    gluDeleteQuadric(pole);
    glPopMatrix();
}


// 绘制地球外框
void drawStandBase() {
    glPushMatrix();
```

```
    glRotatef(-10, 0, 1, 0);
    glRotatef(50, 0, 0, 1);
    //glRotatef(-30, 0, 0, 1);
    GLUquadric* base = gluNewQuadric();
    gluDisk(base, radius + 20, radius + 50, 20, 20);
    gluDeleteQuadric(base);
    glPopMatrix();
}


// 绘制场景
void drawScene() {
    // 设置背景颜色为白色
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    glPopMatrix();

    glPushMatrix();
    // 实现缩放
    glScalef(position[0], position[1], position[2]);
    // 向后移动地球以避免放大时出现黑色圆圈
    glTranslatef(400, 240, -200);
    // 实现自动旋转
    glRotatef(-30, 0, 0, 1);
    drawStandBase();
    drawStandPole();
    // 实现拖动旋转
    glRotatef(rotationX / 100, 0, 1, 0);
    glRotatef(-rotationY / 100, 1, 0, 0);

    // 实现自动旋转
    glRotatef(angle, 0, 1, 0);


    // 绘制支架部分
    //drawStandBase();
    //drawStandPole();

    // 绘制地球
    drawEarth();


    glPopMatrix();

    glutSwapBuffers();
}


// 实现自动旋转效果
void update(int value) {
    angle += 2.0f;
    if (angle > 360) {
```

```
        angle -= 360;
    }

    // 重画
    glutPostRedisplay();

    // 60FPS
    glutTimerFunc(16, update, 0);
}


// 处理鼠标拖动旋转事件
void handleMouseMotion(int x, int y)
{
    int rotationXDelta = x - oldMouseX;
    int rotationYDelta = 480 - y - oldMouseY;
    rotationX += rotationXDelta;
    rotationY += rotationYDelta;
    // 重画
    glutPostRedisplay();
}


// 处理鼠标点击事件
void handleMouseClick(int button, int state, int x, int y)
{
    static bool done = false;
    if (button == GLUT_LEFT_BUTTON)
    {
        oldMouseX = x;
        oldMouseY = 480 - y;
    }
}


int main(int argc, char** argv) {
    // 初始化 GLUT
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);
    glutInitWindowSize(800, 480);


    // 创建窗口
    glutCreateWindow("地球仪");
    initializeRendering();


    // 设置用户操作
    earthTexture = loadBMPTexture("map.bmp");
    glutDisplayFunc(drawScene);
    glutKeyboardFunc(handleKeyboardInput);
    glutReshapeFunc(handleWindowResize);
    glutMotionFunc(handleMouseMotion);
    glutMouseFunc(handleMouseClick);
```

```
    // 运动效果
    glutTimerFunc(16, update, 0);


    glutMainLoop();
    // 退出时清除渲染资源
    deinitializeRendering();
    return 0;
}
```